

1. MVC Architecture (Model-View-Controller)

Your project structure aligns with the **MVC pattern**, which is commonly used in **Node.js** + **Express** applications:

- **Models (`models/`)** → Defines the database schemas (e.g., `User.js`, `Employee.js`, etc.).
- **Controllers (`controllers/`)** → Handles business logic and processes requests (e.g., `authController.js`, `salaryController.js`).
- **Routes (`routes/`)** → Defines API endpoints and maps them to controllers (e.g., `auth.js`, `employee.js`).
- **Middleware (`middleware/`)** → Contains reusable logic such as authentication (`authMiddleware.js`).
- **Database Configuration (`db/`)** → Manages MongoDB connection (`db.js`).

2. Modular Code Structure

Each feature (auth, employees, departments, etc.) is separated into **individual controllers, models, and routes**, making the codebase:

- ✓ **Easier to maintain**
- ✓ **More scalable**
- ✓ **Reusable across different parts of the application**

3. SOLID Principles

- **S - Single Responsibility Principle (SRP)** → Each file has a clear and distinct purpose.
- **O - Open/Closed Principle (OCP)** → The system is **extensible** (new features can be added without modifying existing code).
- **L - Liskov Substitution Principle (LSP)** → Related modules (e.g., controllers) can be swapped without breaking the app.
- **I - Interface Segregation Principle (ISP)** → The app avoids large, bloated functions by keeping logic separated.
- **D - Dependency Inversion Principle (DIP)** → Uses dependency injection (e.g., `db.js` handles database connection separately).

4. RESTful API Design

The `routes/` folder follows **RESTful API** conventions:

- **GET `/api/employee`** → Fetch all employees
- **POST `/api/auth/login`** → Authenticate a user
- **PUT `/api/department/:id`** → Update a department

- **DELETE** `/api/salary/:id` → Delete salary info

This makes it easier for **frontend apps (React, React Native, etc.)** to interact with the backend.

Conclusion

backend design is **clean, scalable, and well-structured**, following:  **MVC architecture** for separation of concerns

 **SOLID principles** for maintainability

 **RESTful API design** for consistency

Beginner Level

① What is the purpose of the `models/` directory?

Answer: It contains **Mongoose schemas** that define the structure of MongoDB collections.

② Why do we use middleware in Express.js?

Answer: Middleware functions **run before reaching the controller** to handle authentication, validation, or logging.

Intermediate Level

③ How does the `controllers/` directory follow the MVC pattern?

Answer: It contains **business logic** that processes requests and interacts with the `models/` to fetch or modify data.

④ Why is `db.js` placed in a separate `db/` folder instead of inside `index.js`?

Answer: It **separates database connection logic**, making the code **modular and reusable**.

Advanced Level

⑤ How would you improve this architecture for high-traffic applications?

Answer:

- Implement **caching (Redis)** to reduce database queries.
- Use **Load Balancers & Clustering** to handle concurrent requests.

- Optimize **database indexing** for better query performance.

⑥ What are the benefits of using a modular routing system (**routes/** folder)?

Answer:

- **Easier debugging** (each route is separate).
- **Scalability** (new routes can be added without breaking existing ones).
- **Better maintenance** (specific logic is contained in individual route files).

Index.js

Express.js API is well-structured, but could benefit from **security and scalability enhancements**.

① Import Required Modules

javascript

```
import express from 'express'  
import cors from 'cors'
```

- **express** → Web framework for handling requests & responses.
- **cors** → Enables Cross-Origin Resource Sharing (CORS) for API access from different domains.

② Import Routes

javascript

```
import authRouter from './routes/auth.js'  
import departmentRouter from './routes/department.js'  
import employeeRouter from './routes/employee.js'  
import salaryRouter from './routes/salary.js'  
import leaveRouter from './routes/leave.js'  
import settingRouter from './routes/setting.js'  
import dashboardRouter from './routes/dashboard.js'
```

- These imported **route handlers** organize API logic for different modules:
 - ✓ Authentication (/api/auth)
 - ✓ Departments (/api/department)
 - ✓ Employees (/api/employee)
 - ✓ Salaries (/api/salary)

- ✓ Leave Management (/api/leave)
- ✓ Settings (/api/setting)
- ✓ Dashboard Analytics (/api/dashboard)

③ Connect to Database

```
javascript
import connectToDatabase from './db/db.js'
connectToDatabase()
```

- Calls `connectToDatabase()` to establish a **MongoDB** connection.

④ Initialize Express App & Middlewares

```
javascript
CopyEdit
const app = express()
app.use(cors()) // Enables CORS for API access
app.use(express.json()) // Parses incoming JSON requests
app.use(express.static('public/uploads')) // Serves static files
(e.g., user profile images)
```

- `express.json()` → Allows API to **parse JSON data** from client requests.
 - `express.static('public/uploads')` → Serves uploaded files (e.g., profile pictures).
-

⑤ Setup API Routes

```
javascript
CopyEdit
app.use('/api/auth', authRouter)
app.use('/api/department', departmentRouter)
app.use('/api/employee', employeeRouter)
app.use('/api/salary', salaryRouter)
```

```
app.use('/api/leave', leaveRouter)
app.use('/api/setting', settingRouter)
app.use('/api/dashboard', dashboardRouter)
```

- Each route is **mounted on a base path**, meaning:
 - Requests to `/api/auth/login` are handled by `authRouter`.
 - Requests to `/api/department/add` are handled by `departmentRouter`, etc.
-

6 Start the Server

javascript

CopyEdit

```
app.listen(process.env.PORT, () => {
  console.log(`Server is Running on port ${process.env.PORT}`)
})
```

- Starts the server on the port **defined in environment variables** (`.env`).
- The **server is now live**, ready to handle API requests.

♦ Easy Level

1 What is the purpose of `express.json()` middleware in this code?

✓ **Answer:** It parses incoming request bodies as JSON, allowing the backend to handle JSON payloads.

② Why do we use `cors()` in this Express application?

✓ Answer: It enables Cross-Origin Resource Sharing (CORS), allowing the API to be accessed from different domains (e.g., frontend running on a different server).

③ What does `connectToDatabase()` do?

✓ Answer: It establishes a connection to MongoDB to enable database operations.

♦ Medium Level

④ How does the app structure follow the MVC pattern?

✓ Answer:

- `routes/` → Acts as the **Controller** (Handles API requests).
- `models/` → Represents the **Model** (Handles database operations).
- `db/db.js` → Manages the **Database Connection**.
- `server.js` → Configures middleware and sets up API endpoints.

⑤ What is the role of `express.static('public/uploads')`?

✓ Answer: It serves static files (like uploaded profile images) from the `public/uploads` directory.

♦ Hard Level

⑥ What are some potential security vulnerabilities in this code?

✓ Answer:

- **No rate limiting** → API is vulnerable to brute-force attacks.
- **No input validation** → Malicious inputs can cause SQL/NoSQL injection.
- **No error-handling middleware** → Errors might expose sensitive information.

⑦ How would you improve the application's scalability?

✓ Answer:

- **Use a Load Balancer** (e.g., Nginx) for handling high traffic.
- **Implement Redis caching** to reduce database queries.
- **Use worker threads** (e.g., BullMQ for background tasks).

USerSEED.js

This script is useful for **seeding an admin account**, but **improvements** are needed for **security and efficiency**.

1 Import Required Modules

```
import User from './models/User.js' // User model (MongoDB schema)
import bcrypt from 'bcrypt' // Library for hashing passwords
import connectToDatabase from './db/db.js' // Connects to MongoDB
```

- `User` → Represents the user model (MongoDB).
 - `bcrypt` → Used to **encrypt** passwords before storing them.
 - `connectToDatabase` → Function to connect to **MongoDB**.
-

2 Define the `userRegister` function

```
javascript
CopyEdit
const userRegister = async () => {
  connectToDatabase() // Establish DB connection
```

- Calls `connectToDatabase()` to ensure MongoDB is **connected**.
-

3 Hash the Password

```
javascript
CopyEdit
const hashPassword = await bcrypt.hash("admin", 10)
```

- `"admin"` → **Raw password**
 - `10` → **Salt rounds** (defines hashing strength).
 - `bcrypt.hash()` → Converts plain text into a **secure hashed password**.
-

4 Create a New Admin User

```
javascript
CopyEdit
const newUser = new User({
```

```
    name: "Admin",
    email: "admin@gmail.com",
    password: hashPassword,
    role: "admin"
  })
  await newUser.save()
```

- **Creates a new user** with name, email, hashed password, and role `"admin"`.
 - **Saves it in MongoDB** using `.save()`.
-

5 Handle Errors

```
javascript
CopyEdit
catch(error) {
  console.log(error)
}
```

- Catches and **logs errors** if something goes wrong (e.g., DB connection failure).
-

6 Call the Function

```
javascript
CopyEdit
userRegister();
```

- This **runs the function immediately** to create an admin user.
-

Interview Questions (Easy → Hard)

♦ Easy Level

1 Why do we use `bcrypt.hash()` before saving the password?

✓ **Answer:** To securely store passwords and protect against data breaches.

2 What does `.save()` do in `newUser.save()`?

✓ **Answer:** It **stores** the new user in the MongoDB database.

♦ Medium Level

③ Why do we pass **10** as the second argument to **bcrypt.hash()**?

✓ **Answer:** It defines the **salt rounds**, making the hash more secure but computationally expensive.

④ What happens if **connectToDatabase()** fails?

✓ **Answer:** The script will throw an error, and it will be caught inside the **catch** block.

♦ Hard Level

⑤ How can we modify this function to prevent duplicate admin creation?

✓ **Answer:** Before creating the user, we should check if an admin **already exists** using:

javascript
CopyEdit

```
const existingUser = await User.findOne({ email:
"admin@gmail.com" });
if (existingUser) return console.log("Admin already exists");
```

⑥ What security risks are present in this script?

✓ **Answer:**

- **No input validation** → Hardcoded email can be a security risk.
- **No environment variables** → Password should not be **hardcoded**.

.ENV

```
PORT=5000 MONGODB_URL="mongodb://127.0.0.1:27017/employeemanagement"
JWT_KEY="employeemanagementSystem7122000"
EMAIL_USER="harshalbisen712@gmail.com" EMAIL_PASSWORD="tqqpecnxcahubef"
```

➤① **Never push **.env** files to GitHub or share them online.**

- **Add **.env** to **.gitignore**** to prevent it from being committed.

bash
CopyEdit

```
# .gitignore
.env
```

② Use **dotenv** package to load environment variables in Node.js.

Install it:

nginx

CopyEdit

```
npm install dotenv
```

•

Load it in your code:

javascript

CopyEdit

```
import dotenv from 'dotenv';
dotenv.config();
```

•

③ Use environment variables in production.

- Instead of storing credentials in **.env**, use a **secrets manager** like:
 - **AWS Secrets Manager**
 - **Google Secret Manager**
 - **Environment Variables** (**process.env.JWT_KEY**)

④ Change these credentials immediately.

- **Rotate your MongoDB credentials** (change the password and update **MONGODB_URL**).
- **Generate a new JWT_KEY** to invalidate any existing tokens.
- **Use an app password instead of your real email password.**

user.js

Import Mongoose

javascript

CopyEdit

```
import mongoose from "mongoose";
```

1.

- Mongoose is an **ODM (Object-Document Mapper)** for MongoDB.

- It helps define and interact with MongoDB documents as JavaScript objects.

Define User Schema

javascript

CopyEdit

```
const userSchema = new mongoose.Schema({
```

2.

- A **schema** defines the structure of documents stored in the `users` collection.

Schema Fields

javascript

CopyEdit

```
name: { type: String, required: true },
email: { type: String, required: true },
password: { type: String, required: true },
role: { type: String, enum: ["admin", "employee"], required:
true },
profileImage: { type: String },
createdAt: { type: Date, default: Date.now },
updatedAt: { type: Date, default: Date.now },
```

3.

- **name**, **email**, **password** → Required fields.
- **role** → Must be either "admin" or "employee" (using `enum`).
- **profileImage** → Optional, stores the path to the user's profile picture.
- **createdAt & updatedAt** → Automatically set timestamps.

Create the Mongoose Model

javascript

CopyEdit

```
const User = mongoose.model("User", userSchema);
```

4.

- `User` is a **model** mapped to the `users` collection.

Export the Model

javascript

CopyEdit

```
export default User;
```

5.

- Makes the `User` model available for use in other files.

Possible Interview Questions & Answers

Beginner (Easy)

① **What is Mongoose, and why do we use it in Node.js applications?**

Answer: Mongoose is an **ODM (Object-Document Mapper)** for MongoDB that helps define schemas, enforce validation, and interact with MongoDB in a structured way.

② **What does `enum` do in Mongoose?**

Answer: `enum` restricts a field to predefined values. In this case, `role` can only be `"admin"` or `"employee"`.

Intermediate (Medium)

③ **Why do we use `default: Date.now` for `createdAt` and `updatedAt`?**

Answer: It ensures that every new user document automatically gets a timestamp when created.

④ **Why is the `password` field stored as plain text? How can we improve security?**

Answer: Storing plain text passwords is a security risk. We should **hash** passwords using `bcrypt` before saving them.

Advanced (Hard)

⑤ **How can we add automatic timestamps (`createdAt` & `updatedAt`) without manually setting them?**

Answer: We can enable Mongoose's built-in timestamps feature:

javascript

CopyEdit

```
const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true },
  password: { type: String, required: true },
  role: { type: String, enum: ["admin", "employee"], required:
true },
  profileImage: { type: String }
}, { timestamps: true }); // Automatically adds createdAt &
updatedAt
```

⑥ **How do you handle unique email validation in Mongoose?**

Answer: Add a `unique` constraint in the schema:

javascript
CopyEdit

```
email: { type: String, required: true, unique: true }
```

- This ensures no two users have the same email.

7 What happens if you try to save a user with a missing required field?

Answer: Mongoose will throw a **validation error**, preventing the document from being saved in MongoDB.

authController.js

file handles user authentication in the Employee Management System (EMS). It includes two main functions:

- **login** → Authenticates a user, generates a JWT token, and returns user details.
- **verify** → Returns authenticated user details when a valid token is provided.

Code Breakdown

1 Importing Dependencies

javascript

CopyEdit

```
import jwt from "jsonwebtoken";  
  
import User from "../models/User.js";  
  
import bcrypt from "bcrypt";
```

- **jsonwebtoken (JWT)** → Used to generate authentication tokens.
- **User model** → Retrieves user data from MongoDB.
- **bcrypt** → Used to securely compare hashed passwords.

2 login Function (User Authentication & Token Generation)

javascript

CopyEdit

```
const login = async (req, res) => {  
  try {  
    const { email, password } = req.body;  
    const user = await User.findOne({ email });  
    if (!user) {  
      return res.status(404).json({ success: false, error: "User  
Not Found" });  
    }  
  }  
}
```

- Extracts `email` and `password` from the request body.
 - Searches for the user in the database using `User.findOne({ email })`.
 - If the user is not found, returns a **404** error.
-

javascript

CopyEdit

```
const isMatch = await bcrypt.compare(password,  
user.password);  
  
if (!isMatch) {  
  return res.status(404).json({ success: false, error:  
"Wrong Password" });  
}
```

- Compares the entered password with the hashed password stored in the database using `bcrypt.compare()`.
- If the password is incorrect, returns a **404** error.

javascript

CopyEdit

```
const token = jwt.sign(  
  { _id: user._id, role: user.role },  
  process.env.JWT_KEY,  
  { expiresIn: "10d" }  
);
```

- Generates a **JWT token** containing:
 - `user._id` → Unique user identifier.
 - `user.role` → Role-based access control.
- Signs the token using `process.env.JWT_KEY` with a **10-day expiration**.

javascript

CopyEdit

```
return res  
  .status(200)  
  .json({  
    success: true,  
    token,  
    user: { _id: user._id, name: user.name, role: user.role  
  },  
  });
```

- If authentication is successful, returns:

- `success: true`
 - `token` (for authentication in future requests)
 - `user` details (`_id`, `name`, `role`)
-

3 `verify` Function (Authenticated User Info)

javascript

CopyEdit

```
const verify = (req, res) => {  
    return res.status(200).json({ success: true, user: req.user  
});  
};
```

- Simply returns the authenticated user's details (`req.user`).
 - This function is useful for **protected routes** that require authentication.
-

Interview Questions & Answers

● Beginner Level

1 What does the `login` function do?

✓ It verifies the user's credentials, generates a JWT token, and returns the user's details.

2 Why do we use `bcrypt.compare()`?

✓ To securely compare a user's entered password with the hashed password stored in the database.

● Intermediate Level

3 What does `jwt.sign()` do?

✓ It generates a JWT token containing user information, signed using a secret key (`JWT_KEY`).

④ What happens if `process.env.JWT_KEY` is missing?

✓ JWT generation will fail, preventing users from logging in.

● Advanced Level

⑤ How would you improve this authentication process?

✓ Use refresh tokens, implement multi-factor authentication (MFA), and store JWT securely (e.g., HTTP-only cookies instead of local storage).

⑥ How can you prevent brute-force login attempts?

✓ Use rate limiting (e.g., `express-rate-limit`) to block repeated failed login attempts from the same IP address.

⑦ Why is storing JWT in local storage not recommended?

✓ Local storage is vulnerable to XSS attacks. Using HTTP-only cookies is a more secure approach.

db.js

The `db.js` file is responsible for connecting the application to the MongoDB database using **Mongoose**.

Code Breakdown

① Importing Mongoose

javascript

CopyEdit

```
import mongoose from "mongoose";
```

- **Mongoose** is an Object Data Modeling (ODM) library for MongoDB, allowing us to interact with the database easily.
-

② Defining the `connectToDatabase` Function

javascript

CopyEdit

```
const connectToDatabase = async () => {  
  try {  
    await mongoose.connect(process.env.MONGODB_URL)  
  } catch(error) {  
    console.log(error)  
  }  
}
```

- Defines an **asynchronous function** to connect to MongoDB.
 - Uses `mongoose.connect(process.env.MONGODB_URL)` to establish the connection.
 - `process.env.MONGODB_URL` is stored in the `.env` file (e.g., `mongodb://127.0.0.1:27017/employeemanagement`).
 - If an error occurs, it is caught in the `catch` block and logged to the console.
-

3 Exporting the Function

javascript

CopyEdit

```
export default connectToDatabase;
```

- Exports the function so it can be used in `server.js` or other files.
-

Interview Questions & Answers

🟢 Beginner Level

1 What is the purpose of `db.js`?

✅ It establishes a connection to the MongoDB database using Mongoose.

❏ **Why do we use `process.env.MONGODB_URL` instead of hardcoding the connection string?**

✅ For security and flexibility—storing it in an environment variable prevents exposing sensitive information.

🟡 Intermediate Level

❏ **What happens if the database connection fails?**

✅ The error is caught in the `catch` block and logged to the console.

❏ **How can we improve error handling in `connectToDatabase()`?**

✅ We can add `console.error("Database connection failed:", error.message)` and exit the process using `process.exit(1)` to stop the server if the database connection fails.

🔴 Advanced Level

❏ **How can you ensure database reconnection in case of failure?**

✅ Use Mongoose event listeners (`mongoose.connection.on("disconnected", callback)`) to automatically attempt reconnection.

❏ **How can you handle connection pooling for scalability?**

✅ Pass options to `mongoose.connect()`, such as `{ useNewUrlParser: true, useUnifiedTopology: true, poolSize: 10 }`, to manage multiple connections efficiently.

❏ **What are some alternatives to Mongoose for MongoDB interaction?**

✅ Native MongoDB driver (`mongodb` package) or ODMs like Prisma.

authMiddleware.js

The `authMiddleware.js` file is responsible for **verifying and authenticating users** based on JSON Web Tokens (JWT). It ensures that only authenticated users can access protected routes

❏ **Importing Dependencies**

javascript

CopyEdit

```
import jwt from "jsonwebtoken";  
import User from "../models/User.js";
```

- **jsonwebtoken**: Used to verify JWT tokens.
 - **User model**: Used to fetch user details from the database.
-

2 Defining **verifyUser** Middleware

javascript

CopyEdit

```
const verifyUser = async (req, res, next) => {  
  try {  
    const token = req.headers.authorization.split(' ')[1];
```

- Extracts the **JWT token** from the **Authorization** header (Bearer <token> format).
 - Splits the header and retrieves the token (**index [1]**).
-

3 Checking if Token is Provided

javascript

CopyEdit

```
if (!token) {  
  return res.status(404).json({ success: false, error: "Token  
Not provided" });  
}
```

- If no token is found, the request is denied with a 404 error.
-

4 Verifying the JWT Token

javascript

CopyEdit

```
const decoded = jwt.verify(token, process.env.JWT_KEY);

if (!decoded) {

    return res.status(404).json({ success: false, error: "Invalid Token" });

}
```

- The token is **verified** using the secret key (JWT_KEY from .env).
 - If the token is invalid, the request is rejected.
-

5 Fetching the User from the Database

javascript

CopyEdit

```
const user = await User.findById({ _id: decoded._id
}).select('-password');

if (!user) {

    return res.status(404).json({ success: false, error: "User not found" });

}
```

- Finds the user by **_id** extracted from the decoded token.

- **Excludes the password field** (`.select('-password')`).
 - If the user **does not exist**, it returns a `404` error.
-

6 Attaching User Data to `req` and Proceeding

javascript

CopyEdit

```
req.user = user;  
  
next();
```

- Attaches the **authenticated user** to `req.user`.
 - Calls `next()` to pass control to the next middleware or controller.
-

7 Handling Errors

javascript

CopyEdit

```
catch (error) {  
  
    return res.status(500).json({ success: false, error: "server  
error" });  
  
}
```

- Catches **server errors** and sends a `500` response.
-

Interview Questions & Answers

● Beginner Level

❏ **1 What is the purpose of `authMiddleware.js`?**

✅ It verifies JWT tokens and ensures that only authenticated users can access protected routes.

❏ **2 What happens if the token is missing in the request?**

✅ The server responds with `{ success: false, error: "Token Not provided" }` and a `404` status code.

🟡 **Intermediate Level**

❏ **3 How does the middleware extract the token from the request?**

✅ It retrieves it from `req.headers.authorization`, which is expected to be in `Bearer <token>` format.

❏ **4 Why do we exclude the password field when fetching the user?**

✅ For **security reasons**, so that hashed passwords are never exposed in the response.

🔴 **Advanced Level**

❏ **5 What would happen if the JWT secret key (`JWT_KEY`) is compromised?**

✅ An attacker could generate valid tokens, allowing unauthorized access. A new key must be generated, and all tokens should be invalidated.

❏ **6 How can we improve this middleware for better security?**

✅ We can check the token's **expiration date** (`exp` field in the JWT payload) and refresh tokens before they expire.

❏ **7 How can we handle token blacklisting for logged-out users?**

✅ Implement a **token blacklist** using Redis or a database to store revoked tokens.

`Department.js`

This file defines a **Mongoose schema** for the `Department` collection and includes a **pre-hook** to handle cleanup when a department is deleted.

❏ **1 Department Schema Definition**

javascript

CopyEdit

```
const departmentSchema = new mongoose.Schema({
  dep_name: {type: String, required: true},
  description: {type: String},
  createdAt: {type: Date, default: Date.now},
  updatedAt: {type: Date, default: Date.now}
})
```

- Defines a *Department* with:
 - *dep_name*: Department name (required).
 - *description*: Optional department details.
 - *createdAt* & *updatedAt*: Timestamps.
-

2 Pre-hook for Cleanup Before Deleting a Department

javascript

CopyEdit

```
departmentSchema.pre("deleteOne", {document: true, query: false},
  async function(next) {
```

- **Pre-hook** (`pre("deleteOne")`) ensures that when a department is deleted, all related records in *Employee*, *Leave*, *Salary*, and *User* collections are also removed.
 - `{ document: true, query: false }` ensures it runs for **document-based** deletions (`department.deleteOne()`).
-

3 Fetching Related Employees

javascript

CopyEdit

```
const employees = await Employee.find({department: this._id})

const empIds = employees.map(emp => emp._id)

const userIds = employees.map(emp => emp.userId);
```

- Finds all **employees** in the department being deleted.
 - Extracts:
 - *empIds*: Employee *_ids* (for removing related salary and leave records).
 - *userIds*: User *_ids* associated with employees.
-

4 Deleting Related Data

javascript

CopyEdit

```
await Employee.deleteMany({department: this._id})

await Leave.deleteMany({employeeId: {$in : empIds}})

await Salary.deleteMany({employeeId: {$in : empIds}})

await User.deleteMany({ _id: { $in: userIds } });
```

- Deletes:
 - Employees belonging to the department.
 - Leave and salary records linked to deleted employees.
 - Users associated with deleted employees.
-

5 Moving to the Next Middleware

javascript

CopyEdit

`next()`

- Calls `next()` to proceed after cleanup.

Interview Questions & Answers

Beginner Level

1 What is the purpose of this `Department.js` file?

✓ It defines the `Department` schema and ensures related records are deleted when a department is removed.

2 What happens if we delete a department?

✓ All employees, their leave records, salaries, and associated user accounts are also deleted.

Intermediate Level

3 Why do we use `deleteOne` instead of `findOneAndDelete`?

✓ `deleteOne` triggers Mongoose middleware properly, whereas `findOneAndDelete` does not support document hooks.

4 What does `{document: true, query: false}` do in `pre("deleteOne")`?

✓ It ensures the middleware runs only when deleting a **specific document** and not a **query-based deletion**.

Advanced Level

5 How can we optimize the deletion process?

✓ Use **transaction-based deletion** (`mongoose.transaction()`) to ensure all deletions either succeed or rollback on failure.

6 What happens if an error occurs during deletion?

✓ The `catch(error) { next(error) }` block ensures the error is passed to the error-handling middleware.

Employee.js

*This file defines a Mongoose schema for storing employee data, linking it to **User** and **Department** models.*

1 Employee Schema Definition

javascript

CopyEdit

```
const employeeSchema = new Schema({

  userId: { type: Schema.Types.ObjectId, ref: "User", required:
true },

  employeeId: { type: String, required: true, unique: true },

  dob: { type: Date },

  gender: { type: String },

  maritalStatus: { type: String },

  designation: { type: String },

  department: { type: Schema.Types.ObjectId, ref:"Department",
required: true },

  salary: { type: Number, required: true },

  createdAt: { type: Date, default: Date.now },

  updatedAt: { type: Date, default: Date.now },

});
```

2 Unique Features

1 *userId* Reference

- *Links the employee to a **User** document (ref: "User").*
- *Ensures that each employee has a corresponding user account.*

2 **employeeId** (Unique Identifier)

- A custom employee ID, different from **_id**.
- Ensures uniqueness to prevent duplicates.

3 **department** Reference

- Links an employee to a **Department** document (**ref: "Department"**).
- Ensures that an employee belongs to a valid department.

4 **salary** Field

- Mandatory numeric field to store employee salary.
-

Interview Questions & Answers

Beginner Level

1 Why do we use **ref: "User"** for **userId**?

✓ To establish a relationship between employees and users, linking them in the database.

2 Why is **employeeId** unique?

✓ To ensure each employee has a distinct identifier, separate from **_id**.

Intermediate Level

3 What is the purpose of referencing the **Department** model in **department**?

✓ To associate an employee with a specific department.

4 What happens if a referenced **User** or **Department** is deleted?

✓ The employee document would still exist but lose its reference, leading to potential data inconsistency.

Advanced Level

5 How can we prevent orphaned employee records if a department is deleted?

✓ Use **pre("deleteOne")** middleware in the **Department.js** model to delete employees related to the department.

6 How can we efficiently retrieve an employee's department details?

✓ Use `.populate("department")` when querying employees to fetch full department details.

Leave.js

This file defines a **Mongoose schema** for storing employee leave requests and tracking their approval status.

1 Leave Schema Definition

javascript

CopyEdit

```
const leaveSchema = new Schema({
  employeeId: { type: Schema.Types.ObjectId, ref: "Employee",
    required: true },
  leaveType: {
    type: String,
    enum: ["Sick Leave", "Casual Leave", "Annual Leave"],
    required: true,
  },
  startDate: { type: Date, required: true },
  endDate: { type: Date, required: true },
  reason: { type: String, required: true },
```

```
status: {  
  type: String,  
  enum: ["Pending", "Approved", "Rejected"],  
  default: "Pending",  
},  
appliedAt: { type: Date, default: Date.now },  
updatedAt: { type: Date, default: Date.now },  
});
```

2 Unique Features

1 **employeeId** Reference

- Connects leave requests to specific employees (*ref: "Employee"*).
- Ensures that only valid employees can apply for leave.

2 **leaveType** Enum

- Restricts leave type to "*Sick Leave*", "*Casual Leave*", or "*Annual Leave*".
- Prevents invalid leave types from being stored.

3 **status** Field (Default: "*Pending*")

- Tracks leave approval process ("*Pending*", "*Approved*", "*Rejected*").
- Default value is "*Pending*" until a manager takes action.

4 **Date** Fields (***startDate***, ***endDate***, ***appliedAt***, ***updatedAt***)

- ***startDate*** & ***endDate***: Define leave duration.
 - ***appliedAt***: Automatically set to the request creation date.
 - ***updatedAt***: Tracks modifications to the leave request.
-

Interview Questions & Answers

Beginner Level

1 Why do we use **ref: "Employee"** for **employeeId**?

✓ To associate a leave request with a specific employee.

2 Why does **leaveType** use **enum**?

✓ To restrict input values and ensure only valid leave types are stored.

Intermediate Level

3 What happens if an employee is deleted?

✓ The leave record would still exist but **lose its reference**, leading to orphaned data.

4 Why does **status** default to **"Pending"**?

✓ To ensure leave requests require **manual approval** before being granted.

Advanced Level

5 How would you prevent employees from applying for leave in the past?

✓ Add validation to ensure **startDate >= current date**.

6 How can we optimize querying leave requests along with employee details?

✓ Use **.populate("employeeId")** when fetching leave records.

Salary.js

This file defines a **Mongoose schema** for storing employee salary details, including basic salary, allowances, deductions, and net salary.

1 **Salary Schema Definition**

javascript

CopyEdit

```
const salarySchema = new Schema({
```

```
    employeeId: { type: Schema.Types.ObjectId, ref: 'Employee',
required: true },

    basicSalary: { type: Number, required: true },

    allowances: { type: Number },

    deductions: { type: Number },

    netSalary: { type: Number },

    payDate: { type: Date, required: true },

    createdAt: { type: Date, default: Date.now },

    updatedAt: { type: Date, default: Date.now },

  });
```

2 Unique Features

1 **employeeId** Reference

- Links salary data to a specific employee using **ref**: "Employee"
- Ensures only valid employees have salary records.

2 **basicSalary** (Required Field)

- Stores the core salary amount.
- Employees **must** have a base salary.

3 **Allowances & Deductions** (Optional Fields)

- **allowances**: Additional earnings (e.g., bonuses, housing, transport).
- **deductions**: Salary reductions (e.g., taxes, penalties).
- Optional because not all employees may have these.

4 **Net Salary** (Automatically Computed in Business Logic)

Should be **calculated** as:

js

CopyEdit

```
netSalary = basicSalary + allowances - deductions;
```

-
- Could be set in a **Mongoose pre-save hook** to ensure correctness.

5 **payDate** Field

- Tracks when the salary was paid.
 - Ensures payroll processing records are stored.
-

Interview Questions & Answers

● Beginner Level

1 Why does **employeeId** reference **Employee**?

✓ To associate salary details with a specific employee.

2 Why is **basicSalary** required but **allowances** and **deductions** are optional?

✓ Every employee must have a base salary, but not all have allowances or deductions.

● Intermediate Level

3 How can we ensure **netSalary** is always correctly calculated?

✓ Use a Mongoose **pre-save middleware** to compute **netSalary** before saving.

4 What happens if an employee is deleted?

✓ The salary record **remains** unless explicitly deleted (cascading delete required).

● Advanced Level

5 How would you optimize salary calculations for payroll reports?

✓ Use **MongoDB aggregation pipelines** to sum salaries, allowances, and deductions efficiently.

6 How would you handle different pay cycles (monthly, bi-weekly, etc.)?

✓ Add a **payCycle** field (enum: ["Monthly", "Bi-Weekly", "Weekly"]) and schedule payments accordingly.

dashboardController.js

This file **aggregates key business metrics** for the **dashboard summary** in the Employee Management System (EMS). It retrieves statistics on employees, departments, salaries, and leave applications.

1 Summary Data Fetching

Total Employees

javascript

CopyEdit

```
const totalEmployees = await Employee.countDocuments();
```

✓ Counts the total number of employees in the system.

Total Departments

javascript

CopyEdit

```
const totalDepartments = await Department.countDocuments();
```

✓ Counts the total number of departments.

2 Salary Aggregation

javascript

CopyEdit

```
const totalSalaries = await Employee.aggregate([
  { $group: { _id: null, totalSalary: { $sum: "$salary" } } } ]
```

])

✓ Uses MongoDB's **aggregation pipeline** to **sum up all employees' salaries**.

✓ `totalSalaries[0]?.totalSalary || 0` ensures that if no salaries exist, the total is 0.

3 Leave Application Summary

Count of Employees Who Applied for Leave

javascript

CopyEdit

```
const employeeAppliedForLeave = await Leave.distinct('employeeId')
```

✓ Uses `.distinct('employeeId')` to get the **unique number of employees** who applied for leave.

Leave Status Breakdown

javascript

CopyEdit

```
const leaveStatus = await Leave.aggregate([
  { $group: { _id: "$status", count: { $sum: 1 } } }
])
```

✓ Groups leave applications by **status** (*Approved, Rejected, Pending*) and counts them.

✓ The **leaveSummary** object extracts and formats the data:

javascript

CopyEdit

```
const leaveSummary = {
  appliedFor: employeeAppliedForLeave.length,
  approved: leaveStatus.find(item => item._id ===
"Approved")?.count || 0,
  rejected: leaveStatus.find(item => item._id ===
"Rejected")?.count || 0,
  pending: leaveStatus.find(item => item._id ===
"Pending")?.count || 0,
}
```

Final Response

javascript

CopyEdit

```
return res.status(200).json({
  success: true,
  totalEmployees,
  totalDepartments,
  totalSalary: totalSalaries[0]?.totalSalary || 0,
  leaveSummary
});
```

 Returns **aggregated dashboard statistics** as a JSON response.

Interview Questions & Answers

● Beginner Level

- 1 What is the purpose of `countDocuments()` in this controller?
✓ It counts the number of documents in a collection (employees, departments, etc.).
 - 2 Why do we use `?.` (optional chaining) in `totalSalaries[0]?.totalSalary || 0`?
✓ It prevents errors when no salary records exist, ensuring a default value of `0`.
-

● Intermediate Level

- 3 Why do we use MongoDB aggregation for salary and leave status instead of simple queries?
✓ Aggregation pipelines efficiently compute grouped statistics directly within the database.
 - 4 How can we optimize performance for large datasets?
✓ Indexes on `salary`, `status`, and `employeeId` would improve query efficiency.
-

● Advanced Level

- 5 How would you modify this function to track salary trends over time?
✓ Add a **date filter** (`$match: {payDate: { $gte: startDate, $lte: endDate }}`) and **group by month/year**.
- 6 What happens if an employee is deleted but they had an applied leave?
✓ Their leave records might remain. We may need **cascading deletes** or an **archive mechanism**.

DepartmentController.js

This file handles CRUD operations for **departments** in the Employee Management System (EMS).

1 Fetching All Departments

javascript

CopyEdit

```
const getDepartments = async (req, res) => {  
  try {  
    const departments = await Department.find()  
    return res.status(200).json({ success: true, departments  
  })  
  } catch(error) {  
    return res.status(500).json({ success: false, error:  
"get department server error" })  
  }  
}
```

- ✓ Retrieves all departments using `Department.find()`.
 - ✓ Returns an **array** of departments.
-

2 Adding a New Department

javascript

CopyEdit

```
const addDepartment = async (req, res) => {  
  try {  
    const { dep_name, description } = req.body;  
    const newDep = new Department({ dep_name, description })  
    await newDep.save()  
    return res.status(200).json({ success: true, department:  
newDep })  
  } catch(error) {
```

```
        return res.status(500).json({ success: false, error:
"add department server error" })
    }
}
```

- ✓ Extracts `dep_name` and `description` from `req.body`.
 - ✓ Creates a **new department** and saves it to the database.
-

3 Fetching a Single Department

javascript

CopyEdit

```
const getDepartment = async (req, res) => {
    try {
        const { id } = req.params;
        const department = await Department.findById({ _id: id
    })
        return res.status(200).json({ success: true, department
    })
    } catch(error) {
        return res.status(500).json({ success: false, error:
"get department server error" })
    }
}
```

- ✓ Uses `findById()` to retrieve a department by its `id`.
- ✓ Returns the **department details**.

4 Updating a Department

javascript

CopyEdit

```
const updateDepartment = async (req, res) => {
  try {
    const { id } = req.params;
    const { dep_name, description } = req.body;
    const updateDep = await Department.findByIdAndUpdate(
      { _id: id }, { dep_name, description }
    )
    return res.status(200).json({ success: true, updateDep
  })
  } catch(error) {
    return res.status(500).json({ success: false, error:
      "edit department server error" })
  }
}
```

- ✓ Finds department by `id` and updates its `dep_name` and `description`.
- ✓ Uses `findByIdAndUpdate()`, but **does not return the updated document** by default.

5 Deleting a Department

javascript

CopyEdit

```
const deleteDepartment = async (req, res) => {  
  try {  
    const { id } = req.params;  
    const deletedep = await Department.findById({ _id: id })  
    await deletedep.deleteOne()  
    return res.status(200).json({ success: true, deletedep  
  })  
  } catch(error) {  
    return res.status(500).json({ success: false, error:  
"delete department server error" })  
  }  
}
```

- ✓ Finds a department by `id`.
 - ✓ Calls `deleteOne()` to remove it from the database.
-

Interview Questions & Answers

● Beginner Level

- 1 What function is used to retrieve all departments?
✓ `Department.find()`
 - 2 What happens if `findById()` doesn't find a department?
✓ It returns `null`, leading to a potential `TypeError`.
-

● Intermediate Level

③ How would you modify **updateDepartment** to return the updated document?

✓ Use `{ new: true }` in `findByIdAndUpdate()`:

javascript

CopyEdit

```
const updateDep = await Department.findByIdAndUpdate(
  { _id: id }, { dep_name, description }, { new: true }
)
```

④ Why do we call **deleteOne()** separately instead of using **findByIdAndDelete()**?

✓ Because **Mongoose middleware** (`pre("deleteOne")`) needs `deleteOne()`, not `findByIdAndDelete()`.

● Advanced Level

⑤ How would you prevent duplicate department names?

✓ Add a **unique index** in the schema:

javascript

CopyEdit

```
dep_name: { type: String, required: true, unique: true }
```

⑥ How can we optimize **getDepartments()** for large datasets?

✓ Use **pagination**:

javascript

CopyEdit

```
const departments = await Department.find().limit(10).skip(10 *
pageNumber)
```

employeeController.js

This file **manages employees** and their **user accounts** in the Employee Management System (EMS). It includes file uploads, user creation, and data retrieval.

① File Upload Handling (multer)

javascript

CopyEdit

```
const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, "public/uploads");
  },
  filename: (req, file, cb) => {
    cb(null, Date.now() + path.extname(file.originalname));
  },
});

const upload = multer({ storage: storage });
```

- ✓ Uses **multer** to handle **file uploads** (profile pictures).
 - ✓ Stores files in **public/uploads/** with a **unique filename** (timestamp + extension).
-

② Add a New Employee

javascript

CopyEdit

```
const addEmployee = async (req, res) => {
  try {
    const {
      name, email, employeeId, dob, gender, maritalStatus,
      designation, department, salary, password, role
    } = req.body;

    const user = await User.findOne({ email });

    if (user) {
      return res.status(400).json({ success: false, error: "user
already registered in emp" });
    }

    const hashPassword = await bcrypt.hash(password, 10);

    const newUser = new User({
      name, email, password: hashPassword, role,
      profileImage: req.file ? req.file.filename : "",
    });

    const savedUser = await newUser.save();
```

```
const newEmployee = new Employee({
  userId: savedUser._id, employeeId, dob, gender,
  maritalStatus, designation, department, salary
});

await newEmployee.save();

return res.status(200).json({ success: true, message:
"employee created" });

} catch (error) {

  return res.status(500).json({ success: false, error: "server
error in adding employee" });

}

};
```

- ✓ **Creates a user account** (`User` collection).
 - ✓ **Creates an employee record** (`Employee` collection).
 - ✓ **Stores hashed passwords** (`bcrypt.hash`).
 - ✓ **Saves profile image filename** (if uploaded).
-

③ Get All Employees

javascript

CopyEdit

```
const getEmployees = async (req, res) => {
  try {
```

```
const employees = await Employee.find()
  .populate("userId", { password: 0 })
  .populate("department");

return res.status(200).json({ success: true, employees });
} catch (error) {
  return res.status(500).json({ success: false, error: "get
employees server error" });
}
};
```

- ✓ Fetches **all employees** from the database.
 - ✓ Uses **.populate()** to include related **userId** (excluding **password**) and **department**.
-

④ Get Employee by ID

javascript

CopyEdit

```
const getEmployee = async (req, res) => {
  const { id } = req.params;
  try {
    let employee;
    employee = await Employee.findById({ _id: id })
      .populate("userId", { password: 0 })
      .populate("department");
```

```
    if (!employee) {
      employee = await Employee.findOne({ userId: id })
        .populate("userId", { password: 0 })
        .populate("department");
    }

    return res.status(200).json({ success: true, employee });
  } catch (error) {
    return res.status(500).json({ success: false, error: "get employees server error" });
  }
};
```

✓ Retrieves an employee by **id**.

✓ Handles cases where the **id** might be a **userId** instead of **employeeId**.

5 Update Employee Details

javascript

CopyEdit

```
const updateEmployee = async (req, res) => {
  try {
    const { id } = req.params;
```

```
    const { name, maritalStatus, designation, department, salary
} = req.body;

    const employee = await Employee.findById({ _id: id });

    if (!employee) {

        return res.status(404).json({ success: false, error:
"employee not found" });

    }

    const user = await User.findById({ _id: employee.userId });

    if (!user) {

        return res.status(404).json({ success: false, error: "user
not found" });

    }

    const updateUser = await User.findByIdAndUpdate({ _id:
employee.userId }, { name });

    const updateEmployee = await Employee.findByIdAndUpdate({
_id: id }, {

        maritalStatus, designation, salary, department

    });

    if (!updateEmployee || !updateUser) {

        return res.status(404).json({ success: false, error:
"document not found" });

    }
```



```
    return res.status(200).json({ success: true, message:
"employee updated" });

    } catch (error) {

        return res.status(500).json({ success: false, error: "update
employees server error" });

    }

};
```

- ✓ Updates both **employee data** and **user data** (name).
 - ✓ Ensures **employee exists before updating**.
-

⑥ Get Employees by Department ID

javascript

CopyEdit

```
const fetchEmployeesByDepId = async (req, res) => {

    const { id } = req.params;

    try {

        const employees = await Employee.find({ department: id });

        return res.status(200).json({ success: true, employees });

    } catch (error) {

        return res.status(500).json({ success: false, error: "get
employeesByDepId server error" });

    }
```

```
};
```

- ✓ Fetches all employees belonging to a **specific department**.
-

Interview Questions & Answers

● Beginner Level

- ① Why do we use **multer** in this code?
 - ✓ To handle **file uploads**, such as profile pictures.
 - ② What happens if the employee's email already exists?
 - ✓ The API returns a **400** error: **"user already registered in emp"**.
-

● Intermediate Level

- ③ Why do we store **userId** in the **Employee** model?
 - ✓ To **link** an employee with their login credentials (**User** collection).
 - ④ What does **.populate("userId", { password: 0 })** do?
 - ✓ Joins employee data with **User** data, but **excludes** the **password** field.
 - ⑤ Why do we hash passwords before saving them?
 - ✓ To ensure **security** and prevent password leaks.
-

● Advanced Level

- ⑥ How can we improve **updateEmployee** to return the updated document?
 - ✓ Use **{ new: true }** in **findByIdAndUpdate()**:

javascript

CopyEdit

```
const updateUser = await User.findByIdAndUpdate({ _id:
employee.userId }, { name }, { new: true });

const updateEmployee = await Employee.findByIdAndUpdate({ _id:
id }, { maritalStatus, designation, salary, department }, { new:
true });
```

7 How would you add pagination to **getEmployees**?

✓ Use **limit()** and **skip()**:

javascript

CopyEdit

```
const page = req.query.page || 1;

const limit = 10;

const employees = await Employee.find()

    .populate("userId", { password: 0 })

    .populate("department")

    .limit(limit)

    .skip((page - 1) * limit);
```

8 How can we enforce file type restrictions on profile images?

✓ Use **fileFilter** in **multer**:

javascript

CopyEdit

```
const upload = multer({

    storage: storage,
```

```
fileFilter: (req, file, cb) => {  
  if (!file.mimetype.startsWith("image/")) {  
    return cb(new Error("Only image files allowed"), false);  
  }  
  cb(null, true);  
}  
});
```

leaveController.js

This file **manages leave requests** for employees, including adding, retrieving, updating, and listing leave records.

1 Add Leave Request

javascript

CopyEdit

```
const addLeave = async (req, res) => {  
  try {  
    const {userId, leaveType, startDate, endDate, reason} =  
req.body;  
    const employee = await Employee.findOne({userId});  
  
    console.log("leave");  
  
    const newLeave = new Leave({  
      employeeId: employee._id, leaveType, startDate,  
endDate, reason  
    });  
  
    await newLeave.save();  
    return res.status(200).json({success: true});  
  } catch (error) {
```

```
        console.log(error.message);
        return res.status(500).json({success: false, error:
"leave add server error"}});
    }
};
```

✓ Finds employee using `userId`.

✓ Creates and **saves a new leave request** linked to `employeeId`.

2 Get Leave Requests for a Specific Employee

javascript

CopyEdit

```
const getLeave = async (req, res) => {
    try {
        const {id, role} = req.params;
        let leaves;
        if (role === "admin") {
            leaves = await Leave.find({employeeId: id});
        } else {
            const employee = await Employee.findOne({userId:
id});
            leaves = await Leave.find({employeeId:
employee._id});
        }

        return res.status(200).json({success: true, leaves});
    } catch (error) {
        console.log(error.message);
        return res.status(500).json({success: false, error:
"leave add .. server error"}});
    }
};
```

✓ Admins can **retrieve leave requests** by `employeeId`.

✓ Regular users fetch their own **leave history** using `userId`.

3 Get All Leave Requests (With Employee Details)

javascript

CopyEdit

```
const getLeaves = async (req, res) => {
  try {
    const leaves = await Leave.find().populate({
      path: 'employeeId',
      populate: [
        { path: 'department', select: 'dep_name' },
        { path: 'userId', select: 'name' }
      ]
    });

    return res.status(200).json({success: true, leaves});
  } catch (error) {
    console.log("Error: ", error.message);
    return res.status(500).json({success: false, error:
"leave add server error"});
  }
};
```

✓ Fetches all leave requests.

✓ Uses **nested .populate()** to get:

- Employee's **department name** (dep_name).
- Employee's **name** from User.

4 Get Leave Request Details by ID

javascript

CopyEdit

```
const getLeaveDetail = async (req, res) => {
  try {
    const {id} = req.params;
    const leave = await Leave.findById({_id: id}).populate({
      path: 'employeeId',
      populate: [
        { path: 'department', select: 'dep_name' },
        { path: 'userId', select: 'name profileImage' }
      ]
    });
  }
};
```

```

    });

    return res.status(200).json({success: true, leave});
  } catch (error) {
    console.log(error.message);
    return res.status(500).json({success: false, error:
"leave detail server error"});
  }
};

```

✓ Fetches **detailed leave info**, including:

- **Employee's department** (`dep_name`).
- **Employee's name and profile image**.

5 Update Leave Status

javascript

CopyEdit

```

const updateLeave = async (req, res) => {
  try {
    const {id} = req.params;
    const leave = await Leave.findByIdAndUpdate({_id: id},
{status: req.body.status});

    if (!leave) {
      return res.status(404).json({success: false, error:
"leave not found"});
    }
    return res.status(200).json({success: true});
  } catch (error) {
    console.log(error.message);
    return res.status(500).json({success: false, error:
"leave update server error"});
  }
};

```

✓ **Updates leave status** (e.g., Approved, Rejected).

✓ **Handles errors** if the leave request does not exist.

Interview Questions & Answers

● Beginner Level

- 1 Why do we store **employeeId** instead of **userId** in the **Leave** model?
✓ To **directly relate** leave records to employees, since employees are linked to users.
 - 2 How does **.populate()** help in **getLeaves**?
✓ It **retrieves additional details** (department name & user info) in a single query.
-

● Intermediate Level

- 3 What happens if an invalid **id** is passed to **getLeaveDetail**?
✓ The API returns a **500** error: **"leave detail server error"**.
- 4 How can we modify **addLeave** to prevent overlapping leave dates?
✓ Add a check before saving:

javascript
CopyEdit

```
const overlappingLeave = await Leave.findOne({
  employeeId: employee._id,
  $or: [
    { startDate: { $lte: endDate }, endDate: { $gte:
startDate } }
  ]
});
if (overlappingLeave) {
  return res.status(400).json({ success: false, error: "Leave
dates overlap" });
}
```

● Advanced Level

- 5 How can we add pagination to **getLeaves**?
✓ Use **limit()** and **skip()**:

javascript
CopyEdit


```
const page = req.query.page || 1;
const limit = 10;
const leaves = await Leave.find()
    .populate("employeeId")
    .limit(limit)
    .skip((page - 1) * limit);
```

⑥ How would you ensure only managers/admins can update leave status?

✓ Add role-based access control before updating:

javascript

CopyEdit

```
if (req.user.role !== "admin" && req.user.role !== "manager") {
    return res.status(403).json({ success: false, error:
    "Unauthorized" });
}
```

salaryController.js

This file **handles employee salary records**, including adding and retrieving salary details.

① Add Salary Record

javascript

CopyEdit

```
const addSalary = async (req, res) => {
    try {
        const {employeeId, basicSalary, allowances, deductions,
        payDate} = req.body;

        const totalSalary = parseInt(basicSalary) +
        parseInt(allowances) - parseInt(deductions);

        const newSalary = new Salary({
            employeeId,
            basicSalary,
            allowances,
            deductions,
```

```

        netSalary: totalSalary,
        payDate
    });

    await newSalary.save();
    return res.status(200).json({success: true});
} catch (error) {
    return res.status(500).json({success: false, error:
"salary add server error"});
}
};

```

- ✓ **Calculates net salary** (basic + allowances - deductions).
 - ✓ **Saves salary details** linked to `employeeId`.
-

2 Get Salary Details

javascript

CopyEdit

```

const getSalary = async (req, res) => {
    try {
        const {id, role} = req.params;

        let salary;
        if(role === "admin") {
            salary = await Salary.find({employeeId:
id}).populate('employeeId', 'employeeId');
        } else {
            const employee = await Employee.findOne({userId:
id});
            salary = await Salary.find({employeeId:
employee._id}).populate('employeeId', 'employeeId');
        }

        return res.status(200).json({success: true, salary});
    } catch (error) {
        return res.status(500).json({success: false, error:
"salary get server error"});
    }
}

```

```
};
```

✓ **Admins** can retrieve salary details using `employeeId`.

✓ **Employees** get their own salary details via `userId`.

Interview Questions & Answers

● Beginner Level

① Why do we calculate `netSalary` instead of storing it directly?

✓ To ensure any changes in **allowances or deductions** are reflected dynamically.

② What does `.populate('employeeId', 'employeeId')` do?

✓ It replaces `employeeId` with the **actual Employee object** but only includes the `employeeId` field.

● Intermediate Level

③ How would you prevent duplicate salary records for the same employee in the same month?

✓ Before saving, check if a salary entry **already exists** for that month:

javascript

CopyEdit

```
const existingSalary = await Salary.findOne({
  employeeId,
  payDate: { $gte: new Date(payDate).setDate(1), $lt: new
Date(payDate).setDate(31) }
});
if (existingSalary) {
  return res.status(400).json({ success: false, error: "Salary
already exists for this month" });
}
```

④ Why does `getSalary` check `role === "admin"`?

✓ To ensure **employees can only see their own salary**, while admins can access all records.

🔴 Advanced Level

5 How would you add pagination to `getSalary`?

✅ Use `limit()` and `skip()`:

javascript

CopyEdit

```
const page = req.query.page || 1;
const limit = 10;
const salary = await Salary.find()
  .populate("employeeId", "employeeId")
  .limit(limit)
  .skip((page - 1) * limit);
```

6 How can we improve salary security so only employees see their own salary?

✅ Implement **authentication & role-based access control (RBAC)** in `getSalary`:

javascript

CopyEdit

```
if (req.user.role !== "admin" && req.user._id.toString() !== id) {
  return res.status(403).json({ success: false, error:
    "Unauthorized" });
}
```

settingController.js

This file **handles user password changes** securely.

🔑 Change Password Function

javascript

CopyEdit

```
const changePassword = async (req, res) => {
  try {
    const {userId, oldPassword, newPassword} = req.body;

    const user = await User.findById({_id: userId});
    if (!user) {
```

```

        return res.status(404).json({success: false, error:
"user not found"});
    }

    const isMatch = await bcrypt.compare(oldPassword,
user.password);
    if (!isMatch) {
        return res.status(400).json({success: false, error:
"wrong old password"});
    }

    const hashPassword = await bcrypt.hash(newPassword, 10);

    await User.findByIdAndUpdate({_id: userId}, {password:
hashPassword});

    return res.status(200).json({success: true});

} catch (error) {
    return res.status(500).json({success: false, error:
"setting error"});
}
};

```

✅ Steps in Change Password:

1. Finds user by **userId**.
2. Compares **oldPassword** with the stored hashed password using `bcrypt.compare()`.
3. If the old password **matches**, it **hashes the new password** and updates it in the database.

Interview Questions & Answers

🟢 Beginner Level

❶ Why do we hash the password before storing it?

- ✅ To prevent plaintext password storage and increase security in case of a database breach.

② Why do we compare **oldPassword** before allowing a change?

- ✓ To ensure the user is **genuinely authorized** to change their password.
-

🟡 Intermediate Level

③ What will happen if the user enters the wrong **oldPassword**?

- ✓ The function returns a **400 status** with **"wrong old password"**.

④ How can we enhance security when updating passwords?

- ✓ Enforce **strong password policies** and **require re-authentication** before changes.
-

🔴 Advanced Level

⑤ How would you prevent brute-force attacks on password changes?

- ✓ Implement a **rate limiter**:

```
javascript
CopyEdit
import rateLimit from 'express-rate-limit';

const passwordChangeLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 5,
  message: "Too many password attempts. Try again later.",
});
```

Apply it to routes:

```
javascript
CopyEdit
app.post("/change-password", passwordChangeLimiter,
changePassword);
```

⑥ How can we ensure that a user is logged in before changing the password?

- ✓ Use **JWT authentication middleware**:

```
javascript
CopyEdit
const authMiddleware = (req, res, next) => {
  const token = req.header("Authorization");
```

```

    if (!token) return res.status(401).json({success: false,
error: "Unauthorized"});
    try {
        const decoded = jwt.verify(token,
process.env.JWT_SECRET);
        req.user = decoded;
        next();
    } catch (error) {
        res.status(401).json({success: false, error: "Invalid
token"});
    }
};

```

Then, apply it:

```
app.post("/change-password", authMiddleware, changePassword);
```

auth.js

1 Login Route (POST /login)

- Calls the `login` function from `authController.js`
- Handles **user authentication** and **JWT token issuance**

2 Verify Route (GET /verify)

- Protected route using `authMiddleware`
- Calls `verify` from `authController.js`
- **Checks if the user is authenticated** and **validates their JWT token**



Code Breakdown

javascript

CopyEdit

```

import express from 'express'
import { login, verify } from '../controllers/authController.js'
import authMiddleware from '../middleware/authMiddleware.js'

```

```
const router = express.Router()

router.post('/login', login) // Handles user login
router.get('/verify', authMiddleware, verify) // Protected route

export default router;
```

Interview Questions & Answers

● Beginner Level

1 What does `router.post('/login', login)` do?

✓ It handles login requests by calling the `login` function from `authController.js`.

2 Why do we use `authMiddleware` in `/verify`?

✓ To restrict access to verified users and prevent unauthorized access.

● Intermediate Level

3 What happens if an unauthenticated user tries to access `/verify`?

✓ `authMiddleware` will reject the request with a 401 Unauthorized error.

4 How does JWT work in authentication?

✓ When a user logs in:

- A **JWT token** is created and sent to the client.
 - The client stores it (e.g., **localStorage** or **HTTP-only cookies**).
 - On **protected routes**, the token is sent in the `Authorization` header.
 - The server **verifies the token** in `authMiddleware`.
-

● Advanced Level

5 How can we improve security in authentication?

✓ **Best Practices:**

- Use **HTTP-only & Secure cookies** for storing tokens.
- Implement **refresh tokens** to prevent frequent logins.
- Use **rate limiting** to prevent brute-force attacks.

⑥ How would you modify `/verify` to return user details if valid?

✓ Modify `verify` function in `authController.js`:

javascript

CopyEdit

```
const verify = async (req, res) => {  
    return res.status(200).json({ success: true, user: req.user  
});  
};
```

department.js

This file defines a **protected API endpoint** for retrieving **dashboard summary data**.

📌 Route in `department.js`

① GET `/summary`

- **Protected Route** → Uses `authMiddleware` to restrict access.
 - Calls `getSummary` from `dashboardController.js`.
 - Returns **dashboard summary data** (e.g., total employees, departments, leave stats).
-

🔑 Code Breakdown

javascript

CopyEdit

```
import express from 'express'  
import authMiddleware from '../middleware/authMiddleware.js'  
import { getSummary } from  
    '../controllers/dashboardController.js';  
  
const router = express.Router()  
  
router.get('/summary', authMiddleware, getSummary) // Protected  
route  
  
export default router;
```

Interview Questions & Answers

🟢 Beginner Level

1 What does `/summary` return?

✓ Aggregated data for the dashboard (e.g., employee count, salary stats).

2 Why is `authMiddleware` used here?

✓ To restrict access to authorized users only.

🟡 Intermediate Level

3 What improvements can be made to `/summary`?

✓ Cache results (e.g., Redis) to reduce database load.

4 How can we modify this to return summary data by department?

✓ Add a query parameter like `?departmentId=xyz` and filter in `getSummary`.

🔴 Advanced Level

5 How can we handle role-based access to this summary?

✓ Modify `authMiddleware` to check user roles (e.g., only managers can access).

`settings.js`

📌 Route in `settings.js`

1 PUT `/change-password`

- **Protected Route** → Uses `authMiddleware` to ensure only authenticated users can access.
 - Calls `changePassword` from `settingController.js`.
 - Updates user password after verifying the old password.
-

🔑 Code Breakdown

javascript
CopyEdit

```
import express from 'express'
```

```
import authMiddleware from '../middleware/authMiddleware.js'
import { changePassword } from
'../controllers/settingController.js'

const router = express.Router()

router.put('/change-password', authMiddleware, changePassword)
// Secure password change route

export default router
```

Interview Questions & Answers

● Beginner Level

① Why do we use **authMiddleware** here?

✓ To ensure only **logged-in users** can change their password.

② What HTTP method is used, and why?

✓ **PUT** because password change **modifies an existing resource** (the user's password).

● Intermediate Level

③ What security measures are applied in **changePassword**?

✓ Verifies old password, hashes new password, and stores it securely.

④ What happens if the old password is incorrect?

✓ The request fails with a **404** response (**wrong old password**).

● Advanced Level

⑤ How can this be improved for better security?

✓ Implement **rate limiting** (to prevent brute-force attacks).

✓ Add **email or SMS verification** before changing the password.

employee.js

This file **defines API routes** for handling employee-related operations, ensuring they are protected with authentication.

Routes in `employee.js`

- ① **GET /** → Fetch all employees (Protected)
 - ② **POST /add** → Add a new employee with an image upload (Protected)
 - ③ **GET /:id** → Get details of a specific employee by ID (Protected)
 - ④ **PUT /:id** → Update an employee's details (Protected)
 - ⑤ **GET /department/:id** → Fetch employees by department ID (Protected)
-

Code Breakdown

javascript
CopyEdit

```
import express from 'express'
import authMiddleware from '../middleware/authMiddleware.js'
import { addEmployee, upload, getEmployees, getEmployee,
updateEmployee, fetchEmployeesByDepId } from
'../controllers/employeeController.js'

const router = express.Router()

router.get('/', authMiddleware, getEmployees) // Fetch all
employees
router.post('/add', authMiddleware, upload.single('image'),
addEmployee) // Add employee with image upload
router.get('/:id', authMiddleware, getEmployee) // Get employee
by ID
router.put('/:id', authMiddleware, updateEmployee) // Update
employee by ID
router.get('/department/:id', authMiddleware,
fetchEmployeesByDepId) // Get employees by department ID

export default router
```

Interview Questions & Answers

● Beginner Level

1 Why is **authMiddleware** used in all routes?

✓ To ensure **only authenticated users** can access employee data.

2 Why is **upload.single('image')** used in **POST /add**?

✓ To allow employees to **upload a profile image** when adding a new record.

● Intermediate Level

3 What would happen if **authMiddleware** was removed?

✓ Unauthorized users could **add, update, or retrieve** employee data.

4 Why does **PUT /:id** require **authMiddleware**?

✓ To prevent **unauthorized modifications** to employee records.

● Advanced Level

5 How can we improve security for file uploads?

✓ Restrict accepted file types (**only images** like **.jpg, .png**).

✓ Set a **file size limit** to prevent large file uploads.

6 What is a potential performance bottleneck in **GET /department/:id**?

✓ If many employees exist in a department, fetching all at once **could slow down** response times.

✓ Use **pagination** or **limit results** to optimize queries.